

```
--snip--
Returning ['JJJJ', 'JJJP', 'JJJB', 'JJJ1', 'JJJ2', 'JJJ3',
'JJPJ', 'JJPP', 'JJPB', 'JJP1', 'JJP2', 'JJP3', 'JJBJ', 'JJBP',
'JJBB', 'JJB1', 'JJB2', 'JJB3', 'JJ1J', 'JJ1P', 'JJ1B', 'JJ11',
'JJ12', 'JJ13', 'JJ2J', 'JJ2P', 'JJ2B', 'JJ21', 'JJ22', 'JJ23',
'JJ3J', 'JJ3P', 'JJ3B', 'JJ31', 'JJ32', 'JJ33', 'JPJJ',
--snip--
```

---

The `getPermsWithRep()` function has a `prefix` string argument that begins as a blank string by default. When the function is called, it first checks for the base case ❶. If `permLength`, the length of the permutations, is 0, an array with `prefix` is returned.

Otherwise, in the recursive case, for each character in the `chars` argument the function creates a new prefix ❷ to pass to the recursive `getPermsWithRep()` call. This recursive call passes `permLength - 1` for the `permLength` argument.

The `permLength` argument starts at the length of the permutations and decreases by one for each recursive call ❸. And the `prefix` argument starts as the blank string and increases by one character for each recursive call. So by the time the base case of `k == 0` is reached, the `prefix` string is the full permutation length of `k`.

For example, let's consider the case of calling `getPermsWithRep('ABC', 2)`. The `prefix` argument defaults to the blank string. The function makes a recursive call with each character of `ABC` concatenated to the blank prefix string as the new prefix. Calling `getPermsWithRep('ABC', 2)` makes these three recursive calls:

- `getPermsWithRep('ABC', 1, 'A')`
- `getPermsWithRep('ABC', 1, 'B')`
- `getPermsWithRep('ABC', 1, 'C')`

Each of these three calls will make its own three recursive calls, but will pass 0 for `permLength` instead of 1. The base case occurs when `permLength == 0`, so these return their prefixes. This is how all nine of the permutations are generated. The `getPermsWithRep()` function generates permutations of larger sets the same way.